

Worldline Global Services — Tailored SDET Interview Preparation

Swapnil Bodade | 13+ Years | SDET / Test Automation Lead | Java + Automation + API + CI/CD + DevOps

Purpose

This guide is tailored by cross-mapping the uploaded Worldline SDET JD against the uploaded resume. The resume shows 13+ years of Quality Engineering experience, current Test Lead work at Xoriant, prior SDET Lead experience at Synechron, banking/financial-services experience at TCS, and skills including Java, JavaScript/TypeScript, Playwright, Selenium, WebdriverIO, Appium, REST Assured, Postman, Jenkins, GitHub Actions, Docker, Kubernetes, AWS, JUnit/TestNG/Cucumber and Git/Bitbucket.

■filecite■turn1file1■L8-L25■ ■filecite■turn1file1■L76-L103■

The Worldline JD asks for an SDET who owns quality end-to-end, works in Scrum, builds automation across the testing pyramid, participates in code review/architecture, creates CI/CD quality gates, and has strong Java, SQL, API, framework-design, Jenkins/GitLab, performance/security, Jira/Confluence and Linux skills, with GCP/Kubernetes/Docker/Kafka as advantages.

■filecite■turn1file0■L20-L38■ ■filecite■turn1file0■L42-L68■

Important: The examples are interview practice examples. They are not claims that you personally used every technology shown. Where the JD lists a skill that is not prominent in the resume, the answer explicitly says to position it as conceptual or an area for ramp-up rather than inventing experience.

1. Interview Focus Map

Area	Your resume evidence	JD relevance	Prep priority
Java / OOP / Collections	Java, Selenium, REST Assured, TestNG/JUnit	Java is primary; strong software engineering	Very High
Framework design	Test Lead/SDET Lead; framework work; migration	Deep test framework design	Very High
API	REST Assured, Postman, Playwright/API	REST, XML/JSON, Postman	Very High
CI/CD	Jenkins, GitHub Actions, Docker, Kubernetes, AWS	Jenkins/GitLab + quality gates	Very High
Testing pyramid / strategy	Functional, integration, regression; Agile/Scrum	Unit/component/integration/E2E	Very High
SQL / DB	Testing/automation background; SQL not prominent	Strong SQLs in Oracle	High
Linux / CLI	Not prominent in resume skills	Strong knowledge	High
Performance	Performance testing at Hurix	JMeter/Gatling/Locust	High
Payments	Banking & Financial Services at TCS	Cards Acquiring / payments	High
GCP / Kafka	Not listed; Kubernetes/AWS are listed	Advantage	Medium
Playwright / JS / TS	Current migration and automation	Not explicit in JD but highly relevant to resume	High

The resume explicitly documents the Selenium-to-Playwright migration, AI-assisted automation, Playwright/WebdriverIO/JavaScript/TypeScript, API automation, CI/CD and Agile/Scrum at Xoriant; Jenkins/Docker/Kubernetes/AWS at Synechron; and Java/Selenium/REST Assured/Postman plus banking experience at TCS.

■filecite■turn1file1■L15-L47■

A. Resume + JD Alignment / Opening Questions

Q1. Tell me about yourself.

Interview-ready answer: Frame your answer around the role: 13+ years in Quality Engineering and SDET leadership; hands-on Java, Selenium, Playwright, API, CI/CD and DevOps; current Test Lead role; migration and framework ownership; earlier banking experience.

Practical example:

"I'm an AI-enabled SDET/Test Automation Lead with 13+ years in Quality Engineering. In my current role at Xoriant I lead automation

Q2. Walk me through your current Xoriant project.

Interview-ready answer: Stay strictly within your resume: US healthcare/health insurance, HIX and multi-state enrollment; Selenium-to-Playwright migration; Playwright/WebdriverIO/JS/TS/Cucumber/API; framework improvements; CI/CD; Agile/Scrum.

Practical example:

Explain: domain → application flow at a high level → your responsibilities → framework → CI pipeline → major challenge → mea

Q3. Why are you moving from healthcare automation to Worldline payments?

Interview-ready answer: Connect your existing engineering skills to the role without claiming payments experience you do not have. You do have banking/financial-services experience at TCS, which is directly relevant.

Practical example:

"My current domain is healthcare, but I also spent several years in banking and financial services. The Worldline role combines

Q4. What are your strongest skills for this JD?

Interview-ready answer: Prioritise Java, test automation/framework design, API testing, Jenkins/CI/CD, Docker/Kubernetes/AWS, Selenium/Playwright, SQL/testing lifecycle, Agile/Scrum and leadership. Your resume explicitly lists these areas.

Practical example:

Example mapping: JD Java → your Java; API → REST Assured/Postman; CI/CD → Jenkins/GitHub Actions; DevOps → Docker/Kubernetes/

Q5. Which JD areas are not your strongest hands-on areas?

Interview-ready answer: Be transparent. The JD explicitly asks for performance tools such as JMeter/Gatling/Locust, Oracle/SQL Developer, Linux/CLI, GitLab, and lists GCP/Kafka as advantages. Your resume lists AWS, Jenkins, Docker, Kubernetes, SQL-related testing exposure, but does not list JMeter/Gatling/Locust, GCP or Kafka as hands-on skills.

Practical example:

Good answer: "My strongest hands-on areas are automation, Java, APIs and CI/CD. Performance tooling such as JMeter/Gatling/Locust

Q6. Why should an SDET own quality end-to-end?

Interview-ready answer: The JD defines the SDET as a quality champion who participates from design/code review through deployment and production monitoring, with no handoff to a separate QA team.

Practical example:

Example: During story refinement, identify testability risks; during development, review API contracts; in CI, run unit/component

Q7. What does "quality as an engineering discipline" mean to you?

Interview-ready answer: It means preventing defects rather than only detecting them: testability, automation, code review, contract validation, CI gates, observability and risk-based testing throughout the lifecycle.

Practical example:

Example: Instead of discovering a bad API contract during UI regression, validate the contract at API/component level in CI and

Q8. What is your experience with Agile/Scrum?

Interview-ready answer: Your resume says you collaborated with development and product teams in Agile/Scrum and previously led QA/release activities. Discuss refinement, planning, daily collaboration, review/retrospective and continuous feedback.

Practical example:

Example: For a new story, participate in refinement, identify acceptance criteria and negative scenarios, automate at the appro

Q9. What is your experience leading a team?

Interview-ready answer: You led a 5-member QA team at TCS and have Test Lead/SDET Lead experience. Emphasise technical direction, planning, risk management, framework standards, mentoring and stakeholder communication.

Practical example:

Example: "I divide work by risk and skill, review automation PRs, establish coding/framework standards, unblock engineers, and c

Q10. What would you do in your first 30/60/90 days?

Interview-ready answer: First understand architecture, payment flows, environments and current quality gates; next identify automation/test-pyramid gaps and flaky areas; then propose measurable improvements and implement high-value changes.

Practical example:

30 days: system + pipeline + test suite map. 60 days: address top reliability/coverage gaps. 90 days: establish sustainable quality

B. Java — Primary Language for the JD

Q11. Explain OOP principles with an automation example.

Interview-ready answer: Encapsulation hides implementation details; inheritance reuses behaviour; polymorphism allows a common interface with different implementations; abstraction exposes essential behaviour while hiding details.

Practical example:

```
interface DriverProvider { WebDriver create(); }
class ChromeProvider implements DriverProvider {
    public WebDriver create() { return new ChromeDriver(); }
}
// Test code depends on DriverProvider, not concrete creation.
```

Q12. Abstract class vs interface?

Interview-ready answer: An abstract class can hold state, constructors and both concrete/abstract methods. An interface defines a contract and supports multiple implementation inheritance. Use an interface for capability/contract and an abstract class when sharing base implementation/state.

Practical example:

```
interface Report { void publish(); }
abstract class BaseReport {
    protected String environment;
    BaseReport(String env) { this.environment = env; }
    abstract void publish();
}
```

Q13. Explain compile-time vs runtime polymorphism.

Interview-ready answer: Method overloading is compile-time polymorphism. Method overriding is runtime polymorphism.

Practical example:

```
class APIClient {
    void get(String url) {}
    void get(String url, int timeout) {} // overload
}
class BasePage { void open() {} }
class LoginPage extends BasePage {
    @Override void open() {} // override
}
```

Q14. final vs finally vs finalize?

Interview-ready answer: final is a keyword for constants/non-overridable methods/non-inheritable classes; finally is an exception-handling block; finalize was a legacy GC-related method and should not be used for resource management.

Practical example:

```
final int TIMEOUT = 30;
try { /* test */ }
finally { /* cleanup */ }
```

Q15. HashMap vs Hashtable?

Interview-ready answer: Both map keys to values, but Hashtable is legacy and synchronized; HashMap is generally preferred for non-concurrent use and permits one null key and null values.

Practical example:

```
Map<String, Integer> counts = new HashMap<>();
counts.put("PASS", 10);
counts.put("FAIL", 2);
```

Q16. HashMap vs ConcurrentHashMap?

Interview-ready answer: HashMap is not thread-safe. ConcurrentHashMap supports concurrent access with thread-safe operations and is designed for concurrent use without synchronizing the entire map.

Practical example:

```
ConcurrentHashMap<String, Integer> counters = new ConcurrentHashMap<>();
counters.merge("PASS", 1, Integer::sum);
```

Q17. List vs Set vs Map?

Interview-ready answer: List preserves ordered elements and allows duplicates; Set represents unique elements; Map stores key-value associations.

Practical example:

```
List<String> tests = Arrays.asList("login", "login");
Set<String> unique = new HashSet<>(tests);
Map<String, String> status = new HashMap<>();
status.put("login", "PASS");
```

Q18. ArrayList vs LinkedList?

Interview-ready answer: ArrayList provides fast indexed access and is usually the default list. LinkedList can be efficient for certain insertion/removal patterns but has slower random access.

Practical example:

```
List<String> testNames = new ArrayList<>();
testNames.add("Login");
testNames.add("Payment");
```

Q19. Comparable vs Comparator?

Interview-ready answer: Comparable defines natural ordering inside the class using compareTo. Comparator defines external/custom ordering using compare.

Practical example:

```
class TestCase {
    String name;
    int priority;
}
tests.sort(Comparator.comparingInt(t -> t.priority));
```

Q20. What is a Java Stream?

Interview-ready answer: A Stream provides declarative operations over collections such as filter, map and collect. It is useful for concise test-data transformations.

Practical example:

```
List<String> failed = results.stream()
    .filter(r -> r.getStatus().equals("FAIL"))
    .map(Result::getName)
    .collect(Collectors.toList());
```

Q21. map vs flatMap in Java Streams?

Interview-ready answer: map transforms each element into one result. flatMap transforms each element into a stream and flattens the resulting streams.

Practical example:

```
List<List<String>> groups = Arrays.asList(
    Arrays.asList("A", "B"),
    Arrays.asList("C")
);
List<String> all = groups.stream()
    .flatMap(List::stream)
    .collect(Collectors.toList());
```

Q22. Checked vs unchecked exceptions?

Interview-ready answer: Checked exceptions must be handled or declared. Unchecked exceptions extend RuntimeException and usually indicate programming/runtime conditions.

Practical example:

```
try {
    Files.readString(Path.of("test-data.json"));
} catch (IOException e) {
    throw new RuntimeException("Unable to read test data", e);
}
```

Q23. throw vs throws?

Interview-ready answer: throw actually throws an exception instance. throws declares that a method may propagate specified exceptions.

Practical example:

```
void readConfig() throws IOException {
    throw new IOException("Config missing");
}
```

Q24. String vs StringBuilder vs StringBuffer?

Interview-ready answer: String is immutable. StringBuilder is mutable and generally faster for single-threaded string construction. StringBuffer is synchronized and legacy-oriented for thread-safe mutable strings.

Practical example:

```
StringBuilder sb = new StringBuilder();
for (String step : steps) {
    sb.append(step).append("\n");
}
System.out.println(sb);
```

Q25. What is immutability and why is String immutable?

Interview-ready answer: An immutable object cannot change after construction. String immutability supports safe sharing, caching and predictable behaviour.

Practical example:

```
String s = "PASS";
s.concat("_RESULT");
System.out.println(s); // PASS
s = s.concat("_RESULT"); // new String assigned
```

Q26. Write Java code to find duplicate numbers.

Interview-ready answer: Use a Set for seen values and another Set for duplicates. This is simple and approximately O(n) average time.

Practical example:

```
int[] arr = {1, 2, 3, 2, 4, 3};
Set<Integer> seen = new HashSet<>();
Set<Integer> dup = new LinkedHashSet<>();

for (int n : arr) {
    if (!seen.add(n)) dup.add(n);
}
System.out.println(dup); // [2, 3]
```

Q27. Find the second largest number.

Interview-ready answer: Track largest and second-largest values in one pass instead of sorting.

Practical example:

```
int[] a = {10, 50, 20, 90, 30};
int max = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
for (int n : a) {
    if (n > max) { second = max; max = n; }
    else if (n > second && n != max) second = n;
}
System.out.println(second); // 50
```

Q28. Explain how you would debug a Java automation failure.

Interview-ready answer: First classify whether it is product, environment, data, dependency or automation failure. Inspect stack trace, logs, screenshot/video/network evidence, reproduce locally/CI, isolate the failing layer, then fix the root cause rather than hiding the exception.

Practical example:

Example: NullPointerException in a Page Object → inspect initialization/lifecycle → reproduce → verify fixture/driver lifecycle

C. Selenium / TestNG / Cucumber / Framework Design

Q29. Explain Page Object Model.

Interview-ready answer: POM encapsulates page locators and actions behind page-level/business methods. Tests remain readable and locator changes are centralised.

Practical example:

```
class LoginPage {
    private WebDriver driver;
    private By username = By.id("username");

    LoginPage(WebDriver driver) { this.driver = driver; }

    void login(String user) {
        driver.findElement(username).sendKeys(user);
    }
}
```

Q30. What is PageFactory and what are its disadvantages?

Interview-ready answer: PageFactory initializes @FindBy fields using Selenium's proxy mechanism. A modern framework can prefer explicit By/Locator objects when clearer lifecycle and wait behaviour are desired.

Practical example:

```
@FindBy(id="username")
WebElement username;

public LoginPage(WebDriver driver) {
    PageFactory.initElements(driver, this);
}
```

Q31. What is StaleElementReferenceException and how do you handle it?

Interview-ready answer: It occurs when a previously located element is no longer attached to the current DOM. Prefer locating just before interaction and use a targeted retry only when the DOM replacement is expected.

Practical example:

```
By save = By.id("save");
new WebDriverWait(driver, Duration.ofSeconds(10))
    .until(ExpectedConditions.elementToBeClickable(save))
    .click();
```

Q32. Implicit vs explicit wait?

Interview-ready answer: Implicit wait applies to element lookup globally. Explicit wait waits for a specific condition. In robust frameworks, explicit waits are often preferred for precise synchronisation; avoid unnecessary mixing.

Practical example:

```
WebElement button = new WebDriverWait(driver, Duration.ofSeconds(10))
    .until(ExpectedConditions.elementToBeClickable(By.id("submit")));
button.click();
```

Q33. How do you handle dynamic elements?

Interview-ready answer: Use stable attributes such as data-testid where available, CSS/XPath relationships when necessary, and explicit waits for state rather than fixed sleeps.

Practical example:

```
WebElement row = driver.findElement(
    By.cssSelector("[data-testid='payment-row'][data-id='12345']")
);
```

Q34. findElement vs findElements?

Interview-ready answer: findElement returns the first matching element or throws NoSuchElementException. findElements returns a list; if nothing matches, it returns an empty list.

Practical example:

```
List<WebElement> errors =
    driver.findElements(By.cssSelector(".error"));
if (!errors.isEmpty()) {
    System.out.println("Errors found: " + errors.size());
}
```

Q35. How do you capture a screenshot for a failed TestNG test?

Interview-ready answer: Use a TestNG listener such as ITestListener and capture evidence in onTestFailure. Store it with a unique test name/build identifier.

Practical example:

```
public void onTestFailure(ITestResult result) {
    TakesScreenshot ts = (TakesScreenshot) driver;
    File src = ts.getScreenshotAs(OutputType.FILE);
    // copy to reports/<build>/<test>.png
}
```

Q36. How do you rerun failed TestNG tests?

Interview-ready answer: Use a retry analyzer for controlled transient failures or rerun the failed-test XML/list produced by the framework. Do not use retries to hide deterministic product failures.

Practical example:

```
@Test(retryAnalyzer = RetryAnalyzer.class)
public void paymentTest() {
    // test
}
```

Q37. What are TestNG listeners?

Interview-ready answer: Listeners hook into test lifecycle events such as test start, success, failure and suite events. They are useful for screenshots, logging and reporting.

Practical example:

```
public class Listener implements ITestListener {
    @Override
    public void onTestFailure(ITestResult result) {
        // capture screenshot/logs
    }
}
```

Q38. What is DataProvider in TestNG?

Interview-ready answer: DataProvider supplies multiple data sets to the same test method, enabling data-driven testing.

Practical example:

```
@DataProvider(name = "users")
Object[][] users() {
    return new Object[][] {
        {"qa1", "pass1"},
        {"qa2", "pass2"}
    };
}

@Test(dataProvider = "users")
public void login(String user, String pass) {}
```

Q39. How do you run tests in parallel safely?

Interview-ready answer: Use independent test data, thread-safe driver management, isolated sessions and thread-safe reporting. A common Selenium pattern is ThreadLocal<WebDriver>.

Practical example:

```
private static ThreadLocal<WebDriver> driver = new ThreadLocal<>();

@BeforeMethod
public void setup() {
    driver.set(new ChromeDriver());
}

@AfterMethod
public void tearDown() {
    driver.get().quit();
    driver.remove();
}
```

Q40. How do you design a maintainable automation framework?

Interview-ready answer: Separate test intent from implementation: tests/features → page/service objects → reusable utilities → configuration/data → reporting. Enforce coding standards, stable locators, explicit waits, clean failure handling and CI execution.

Practical example:

```
src/test/java/tests/LoginTest.java
src/main/java/pages/LoginPage.java
src/main/java/api/UserApi.java
src/main/java/utils/WaitUtils.java
src/main/resources/config.properties
```

Q41. What design patterns do you use in automation?

Interview-ready answer: Common patterns include POM, Factory for browser/client creation, Builder for complex test data, Strategy for interchangeable behaviour, and dependency injection/fixtures for lifecycle. Use Singleton cautiously.

Practical example:

```
interface DriverFactory { WebDriver create(); }

class ChromeFactory implements DriverFactory {
    public WebDriver create() { return new ChromeDriver(); }
}
```

Q42. Explain your Selenium-to-Playwright migration.

Interview-ready answer: Your resume states that you led the migration. Explain the reason, scope, locator/page-object strategy, CI integration, flaky-test handling and rollout approach without inventing metrics.

Practical example:

Migration approach:

1. Baseline existing Selenium suite.
2. Identify stable locators and business flows.
3. Implement equivalent Playwright page objects.
4. Remove fixed waits where Playwright auto-waiting handles state.
5. Run old/new suites in parallel during transition.
6. Retire stable Selenium coverage after parity.

D. Playwright + JavaScript/TypeScript

Q43. Why Playwright for modern automation?

Interview-ready answer: Playwright provides browser automation with auto-waiting, multiple browser engines, network interception, tracing and strong test-runner support. Your resume specifically lists Playwright and the Selenium-to-Playwright migration.

Practical example:

```
import { test, expect } from "@playwright/test";

test("login", async ({ page }) => {
    await page.goto("/login");
    await page.getByTestId("username").fill("qa");
    await page.getByTestId("password").fill("secret");
    await page.getByRole("button", { name: "Login" }).click();
    await expect(page).toHaveURL(/dashboard/);
});
```

Q44. How do Playwright locators differ from Selenium locators?

Interview-ready answer: Playwright locators are designed around resilient, user-facing and test-id based selection and include auto-waiting/retry behaviour. Selenium typically exposes WebElement lookup plus explicit wait APIs.

Practical example:

```
const login = page.getByRole("button", { name: "Login" });
await login.click();
```

Q45. How do you design a Playwright Page Object?

Interview-ready answer: Keep Page and Locator fields private, expose business actions, and keep assertions either in the test or in clearly named validation methods.

Practical example:

```
export class LoginPage {
  constructor(private page: Page) {}

  private username = this.page.getByTestId("username");
  private password = this.page.getByTestId("password");

  async login(user: string, pass: string) {
    await this.username.fill(user);
    await this.password.fill(pass);
    await this.page.getByRole("button", {name:"Login"}).click();
  }
}
```

Q46. What are fixtures in Playwright?

Interview-ready answer: Fixtures provide reusable setup and teardown and inject dependencies such as page, context or custom services into tests.

Practical example:

```
test("dashboard", async ({ page }) => {
  // page is provided by the Playwright fixture
  await page.goto("/dashboard");
});
```

Q47. How do you handle API + UI testing in Playwright?

Interview-ready answer: Use APIRequestContext for setup/cleanup or API validation, then use the browser for UI behaviour. This reduces UI dependency for test-data setup.

Practical example:

```
const response = await request.post("/users", {
  data: { name: "Test User" }
});
expect(response.ok()).toBeTruthy();

await page.goto("/users");
```

Q48. How do you intercept a network request?

Interview-ready answer: Use page.route or context.route to inspect, modify, mock or block requests.

Practical example:

```
await page.route("**/api/payments", async route => {
  await route.fulfill({
    status: 200,
    contentType: "application/json",
    body: JSON.stringify({ status: "APPROVED" })
  });
});
```

Q49. How do you debug a flaky Playwright test?

Interview-ready answer: Use trace, screenshot/video, console/network logs, isolate test data, check environment timing and identify whether the failure is product, test or infrastructure related.

Practical example:

```
npx playwright test --trace on
# Then inspect the generated trace for actions,
# DOM snapshots, network and timing.
```

Q50. What is the difference between browser, context and page?

Interview-ready answer: Browser is the browser process; BrowserContext is an isolated browser session/profile; Page is a tab within a context.

Practical example:

```
const browser = await chromium.launch();
const context = await browser.newContext();
const page = await context.newPage();
await page.goto("https://example.com");
```

Q51. How do you run Playwright tests in parallel?

Interview-ready answer: Use worker configuration while ensuring test data and external state are isolated. Do not make tests depend on execution order.

Practical example:

```
// playwright.config.ts
export default defineConfig({
  workers: 4,
  fullyParallel: true
});
```

Q52. How do you manage environment configuration?

Interview-ready answer: Use typed configuration/environment variables and avoid hard-coding secrets. Keep environment-specific base URLs and credentials outside source code.

Practical example:

```
const baseUrl = process.env.BASE_URL ?? "https://qa.example.com";
const user = process.env.TEST_USER;
```

Q53. JavaScript var/let/const — what matters in automation?

Interview-ready answer: Prefer const by default, let when reassignment is needed, and avoid var in modern framework code because of function scoping and hoisting pitfalls.

Practical example:

```
const pageName = "Login";
let retryCount = 0;
retryCount++;
// var is avoided in new framework code.
```

Q54. Promise.all in test automation — where is it useful?

Interview-ready answer: It can run independent asynchronous operations concurrently, but do not use it when operations have dependencies or when parallel requests could violate test-data/business constraints.

Practical example:

```
const [user, config] = await Promise.all([
  getUser(),
  getConfig()
]);
```

Q55. Explain closure with an automation utility.

Interview-ready answer: A closure lets a returned function retain access to private state, useful for counters, caching or configuration.

Practical example:

```
function createRetryCounter() {
  let count = 0;
  return () => ++count;
}
const nextAttempt = createRetryCounter();
console.log(nextAttempt()); // 1
console.log(nextAttempt()); // 2
```

Q56. How would you implement a reusable retry utility?

Interview-ready answer: Retry only operations that are expected to be transient. Capture the final error and preserve evidence. Avoid retrying assertion failures blindly.

Practical example:

```
async function retry(fn, attempts) {
  let lastError;
  for (let i = 1; i <= attempts; i++) {
    try { return await fn(); }
    catch (e) { lastError = e; }
  }
  throw lastError;
}
```

E. API Testing / REST Assured / Postman

Q57. What do you validate in an API test?

Interview-ready answer: Validate HTTP status, headers, schema/structure, mandatory fields, business rules, data persistence, error behaviour, response time where relevant, authentication/authorisation and idempotency.

Practical example:

```
given()
  .when().get("/users/101")
  .then()
  .statusCode(200)
  .body("id", equalTo(101));
```

Q58. Explain GET, POST, PUT, PATCH and DELETE.

Interview-ready answer: GET retrieves data; POST creates/submit a resource or action; PUT generally replaces a resource; PATCH partially updates; DELETE removes a resource.

Practical example:

```
POST /payments
{
  "amount": 1000,
  "currency": "EUR"
}

GET /payments/123
PATCH /payments/123
DELETE /payments/123
```

Q59. Authentication vs authorisation?

Interview-ready answer: Authentication verifies who the caller is. Authorisation determines what the authenticated caller is allowed to do.

Practical example:

```
Authorization: Bearer <token>
GET /merchant/payments
// A valid token may still receive 403 if the role lacks permission.
```

Q60. 401 vs 403?

Interview-ready answer: 401 indicates the request lacks valid authentication credentials. 403 means the server understood the request but refuses access due to permissions/policy.

Practical example:

```
GET /admin
No token -> 401
Valid user token without admin permission -> 403
```

Q61. 400 vs 404 vs 409 vs 500 vs 502?

Interview-ready answer: 400: invalid request; 404: resource not found; 409: conflict; 500: server-side error; 502: gateway/backend communication problem. Always confirm exact API contract.

Practical example:

```
POST /users with invalid JSON -> 400
GET /users/999999 -> 404
Create existing unique user -> 409
```

Q62. What is contract testing?

Interview-ready answer: It verifies that a service's API contract—request/response structure and expectations—remains compatible between producer and consumer.

Practical example:

```
Expected response:
{
  "id": 101,
  "status": "ACTIVE"
}
A contract test fails if "id" changes type from number to string unexpectedly.
```

Q63. How do you validate JSON response data against a database?

Interview-ready answer: Call the API, extract business identifiers, query the database, and compare authoritative fields. Avoid brittle comparisons of timestamps or generated fields unless required.

Practical example:

```
API: paymentId=123, status=APPROVED
SQL:
SELECT status FROM payments WHERE payment_id = 123;
Expected DB status = APPROVED
```

Q64. How would you automate a payment API flow?

Interview-ready answer: Cover positive approval, invalid amount/currency, missing mandatory fields, duplicate/idempotent request, authentication, authorisation, timeout/backend failure and reconciliation/persistence.

Practical example:

```
POST /payments -> 201/200
Repeat same idempotency key -> same business result, not an accidental duplicate
Query payment -> verify persisted status
```

Q65. What is idempotency and why is it important in payments?

Interview-ready answer: An idempotent operation can be repeated without creating unintended additional effects. Payment APIs often use an idempotency key to prevent duplicate charges.

Practical example:

```
POST /payments
Idempotency-Key: abc-123

Retry the same request with the same key:
Expected: same payment result/reference, not two charges.
```

Q66. How do you test negative API scenarios systematically?

Interview-ready answer: Partition inputs into valid/invalid classes, boundary values, missing fields, wrong types, malformed JSON/XML, auth failures, duplicate requests, dependency failures and rate/timeout conditions.

Practical example:

```
amount = -1 -> validation error
currency = "XYZ" -> invalid code
missing merchantId -> mandatory-field error
amount = max + 1 -> boundary error
```

F. SQL / Oracle / Database / Linux CLI

Q67. What SQL topics should an SDET know for this role?

Interview-ready answer: CRUD, joins, aggregation, GROUP BY/HAVING, subqueries, indexes, constraints, transactions, primary/foreign keys, duplicate detection and data validation. The JD explicitly asks for strong SQL/database knowledge and Oracle/SQL Developer.

Practical example:

```
SELECT merchant_id, COUNT(*)
FROM payments
GROUP BY merchant_id
HAVING COUNT(*) > 10;
```

Q68. Primary key vs foreign key?

Interview-ready answer: Primary key uniquely identifies a row and cannot be null. Foreign key references a key in another table to enforce relationship integrity.

Practical example:

```
CREATE TABLE merchant (
  merchant_id NUMBER PRIMARY KEY,
  name VARCHAR2(100)
);

CREATE TABLE payment (
  payment_id NUMBER PRIMARY KEY,
  merchant_id NUMBER REFERENCES merchant(merchant_id)
);
```

Q69. INNER JOIN vs LEFT JOIN?

Interview-ready answer: INNER JOIN returns matching rows from both tables. LEFT JOIN returns all rows from the left table and matching rows from the right, with nulls when no match exists.

Practical example:

```
SELECT m.merchant_id, p.payment_id
FROM merchant m
LEFT JOIN payment p
ON m.merchant_id = p.merchant_id;
```

Q70. Find duplicate records.

Interview-ready answer: Group by the business key and use HAVING COUNT(*) > 1.

Practical example:

```
SELECT merchant_id, transaction_ref, COUNT(*)
FROM payments
GROUP BY merchant_id, transaction_ref
HAVING COUNT(*) > 1;
```

Q71. Find the second-highest salary/value.

Interview-ready answer: Use DENSE_RANK or a subquery depending on whether duplicate values should share a rank.

Practical example:

```
SELECT amount
FROM (
    SELECT amount,
           DENSE_RANK() OVER (ORDER BY amount DESC) rnk
    FROM payments
)
WHERE rnk = 2;
```

Q72. What is an index and when can it help testing/debugging?

Interview-ready answer: An index can speed lookups but adds storage and write overhead. In test data investigation, knowing indexed keys helps explain query performance.

Practical example:

```
CREATE INDEX idx_payment_ref
ON payments(transaction_ref);

SELECT * FROM payments
WHERE transaction_ref = 'TX123';
```

Q73. Explain ACID properties.

Interview-ready answer: Atomicity: all-or-nothing; Consistency: valid state transitions; Isolation: concurrent transactions behave according to isolation rules; Durability: committed data survives failures.

Practical example:

```
Payment transaction:
Debit + ledger update should commit together.
If one required operation fails, transaction handling should prevent partial inconsistent state.
```

Q74. What is a transaction rollback?

Interview-ready answer: Rollback reverses uncommitted changes when a transaction cannot complete successfully.

Practical example:

```
BEGIN;
UPDATE payments SET status='CAPTURED'
WHERE payment_id=123;

-- validation fails
ROLLBACK;
```

Q75. How would you validate API → DB consistency?

Interview-ready answer: Identify the API's business key, call the API, query the corresponding DB record, compare status/amount/reference fields, and account for eventual consistency where applicable.

Practical example:

```
API paymentId = 123
SQL:
SELECT amount, status, reference
FROM payments
WHERE payment_id = 123;
```

Q76. Give Linux commands useful to an SDET.

Interview-ready answer: Common commands include pwd, ls, cd, grep, find, tail, head, cat, less, ps, top, df, du, chmod, curl, netstat/ss and kill. The exact command depends on environment.

Practical example:

```
tail -f application.log
grep -i "ERROR" application.log
ps -ef | grep java
curl -i https://qa.example.com/health
```

Q77. How do you investigate a failing service on Linux?

Interview-ready answer: Check process status, logs, CPU/memory/disk, network connectivity, ports, environment variables and recent deployment/config changes.

Practical example:

```
ps -ef | grep payment-service
tail -n 200 payment.log
df -h
free -m
ss -lntp
```

Q78. What is grep useful for in automation troubleshooting?

Interview-ready answer: It searches logs/output for patterns such as ERROR, transaction IDs or test IDs.

Practical example:

```
grep -i "TX123" application.log
grep -i "timeout\\|exception" application.log
```

G. Jenkins / GitLab / CI-CD / Docker / Kubernetes / AWS

Q79. Explain a CI/CD pipeline for an SDET.

Interview-ready answer: Source checkout → build → unit/component tests → API/integration tests → deploy to test environment → E2E smoke/regression → report → quality gate → release/deploy. The JD explicitly asks for CI/CD quality gates.

Practical example:

```
pipeline {
  stages {
    stage('Build') { steps { sh 'mvn clean test' } }
    stage('API Tests') { steps { sh 'mvn test -Dgroups=api' } }
    stage('UI Tests') { steps { sh 'mvn test -Dgroups=ui' } }
  }
}
```

Q80. Have you created Jenkins pipelines? Explain.

Interview-ready answer: Your resume lists Jenkins and CI/CD automation. Explain pipeline-as-code, parameters, stages, credentials, parallel execution, reports, artifacts and failure gates.

Practical example:

```
pipeline {
  agent any
  stages {
    stage('Checkout') { steps { checkout scm } }
    stage('Test') { steps { sh 'mvn clean test' } }
    stage('Report') { steps { junit 'target/surefire-reports/*.xml' } }
  }
}
```

Q81. What is a quality gate?

Interview-ready answer: A quality gate is an automated condition that must pass before merge/release, such as critical tests, code quality, security checks or coverage thresholds.

Practical example:

```
if (criticalTestsFailed) {
  error("Release blocked: critical quality gate failed");
}
```

Q82. How would you prevent flaky tests from blocking every pipeline?

Interview-ready answer: First classify and measure flakiness, quarantine only with visibility, fix root causes, use targeted retries only for transient infrastructure conditions, and keep critical-path tests strict.

Practical example:

```
Retry network setup once; do not retry a deterministic assertion such as
expected APPROVED but received DECLINED.
```

Q83. Git merge vs rebase?

Interview-ready answer: Merge creates a merge commit and preserves branch history. Rebase rewrites commits onto a new base and produces a linear history. Follow team policy and never rebase shared history casually.

Practical example:

```
git checkout feature
git fetch origin
git rebase origin/main
# resolve conflicts
git push --force-with-lease
```

Q84. What is cherry-pick?

Interview-ready answer: It applies a specific commit from another branch onto the current branch.

Practical example:

```
git checkout release
git cherry-pick abc1234
```

Q85. How do you resolve a Git conflict?

Interview-ready answer: Inspect conflict markers, understand both changes, choose or combine correct code, run tests, stage and continue the operation.

Practical example:

```
git status
# edit file and resolve <<<< ==== >>>>
git add .
git rebase --continue
```

Q86. Docker image vs container?

Interview-ready answer: An image is an immutable packaged template. A container is a running instance of an image.

Practical example:

```
docker build -t sdet-tests .
docker run --rm sdet-tests
```

Q87. Why use Docker for test automation?

Interview-ready answer: It standardises runtime dependencies and creates reproducible environments, useful for CI agents and service dependencies.

Practical example:

```
FROM maven:3.9-eclipse-temurin-21
COPY . /tests
WORKDIR /tests
RUN mvn test
```

Q88. What Kubernetes concepts should an SDET know?

Interview-ready answer: Pod, Deployment, Service, ConfigMap, Secret, namespace, logs, probes and scaling. For testing, know how to inspect deployed services and collect logs.

Practical example:

```
kubectl get pods
kubectl get svc
kubectl logs <pod>
kubectl describe pod <pod>
```

Q89. What is a Kubernetes readiness probe?

Interview-ready answer: It indicates whether a container is ready to receive traffic. A failing readiness probe removes the pod from service endpoints.

Practical example:

```
readinessProbe:
  httpGet:
    path: /health
    port: 8080
```

Q90. How does AWS relate to your automation experience?

Interview-ready answer: Your resume lists AWS and your previous role used Jenkins, Docker, Kubernetes and AWS. Discuss test environments, credentials, logs, infrastructure and CI agents without claiming services not actually used.

Practical example:

Example:
CI agent runs tests → deploys to an AWS-hosted QA environment → executes API/UI tests → publishes reports.

Q91. What is GitLab CI and how is it similar to Jenkins?

Interview-ready answer: Both automate build/test/deploy workflows. Jenkins is highly extensible through plugins and pipeline-as-code; GitLab CI is integrated with GitLab repositories and uses .gitlab-ci.yml.

Practical example:

```
stages:
  - test

api_test:
  stage: test
  script:
    - mvn clean test
```

Q92. How would you run UI and API tests in parallel?

Interview-ready answer: Separate independent suites into pipeline stages/jobs, run them concurrently, collect artifacts, then enforce a final quality gate.

Practical example:

```
parallel jobs:
  API tests -> api-report
  UI tests  -> ui-report
Both must pass before release gate.
```

H. Testing Strategy / Pyramid / Risk / Performance / Security

Q93. Explain the testing pyramid.

Interview-ready answer: Use many fast unit tests, fewer component/integration tests, and a smaller number of end-to-end tests. The exact distribution depends on architecture and risk.

Practical example:

Example:
Unit: validation rules
Component/API: payment service behaviour
Integration: payment + database + external dependency
E2E: merchant checkout journey

Q94. Why not automate everything through UI?

Interview-ready answer: UI tests are slower, more brittle and expensive to diagnose. Business logic can often be validated faster at unit/API/component layers.

Practical example:

```
Payment validation:
Unit -> amount rules
API -> payment contract/business rules
E2E -> only critical customer journey
```

Q95. What does shift-left testing mean?

Interview-ready answer: Move quality activities earlier: requirements/refinement, design, code review, unit/component tests, API contracts and CI checks before expensive E2E/regression.

Practical example:

During story refinement:

Ask: What happens for duplicate payment? What is the timeout? What is the expected status for invalid currency?

Q96. How do you think adversarially as the JD expects?

Interview-ready answer: Start with failure modes: invalid input, duplicate requests, partial failures, timeouts, stale data, concurrency, permissions, retries, dependency outages and boundary values.

Practical example:

Payment:

1. Network timeout after request sent
2. Client retries
3. Backend already processed transaction

Expected: no duplicate charge; idempotency protects the operation.

Q97. How do you decide what to automate?

Interview-ready answer: Prioritise stable, repeatable, high-value, regression-heavy and business-critical scenarios. Avoid automating unstable exploratory checks until requirements stabilise.

Practical example:

Automate:

login, core payment flow, API contracts, regression validations.

Keep:

new exploratory UX investigation as manual/exploratory initially.

Q98. Severity vs priority?

Interview-ready answer: Severity describes impact of a defect; priority describes urgency/order of fixing it. They can differ.

Practical example:

High severity/low priority: rare admin report corruption with workaround.

Low severity/high priority: minor UI issue on a critical launch campaign.

Q99. What is risk-based testing?

Interview-ready answer: Prioritise testing according to likelihood and impact of failure, considering business criticality, technical complexity, change size and historical defects.

Practical example:

Payment authorisation change:

High risk -> deeper API, integration, negative, performance and E2E coverage.

Minor UI text change -> focused UI check.

Q100. Explain equivalence partitioning and boundary value analysis.

Interview-ready answer: Equivalence partitioning divides inputs into representative classes. Boundary analysis tests values at and around limits.

Practical example:

Amount allowed: 1-10000

Equivalence: 0/negative, valid range, >10000

Boundary: 0, 1, 2, 9999, 10000, 10001

Q101. How would you test a login screen?

Interview-ready answer: Cover valid login, invalid credentials, blank fields, lockout/rate limits, password rules, session handling, authorisation, injection/security basics, accessibility and error messaging.

Practical example:

Cases:

valid user -> dashboard

wrong password -> error

locked user -> blocked

expired session -> login

unauthorised role -> 403/denied

Q102. Performance testing: load vs stress vs spike?

Interview-ready answer: Load tests expected traffic; stress tests push beyond expected capacity; spike tests apply sudden traffic changes. Endurance tests sustained load over time.

Practical example:

Expected 1000 TPS:
Load -> 1000 TPS
Stress -> increase until degradation/failure
Spike -> 100 -> 2000 TPS suddenly
Endurance -> 1000 TPS for hours

Q103. How would you approach JMeter performance testing?

Interview-ready answer: Define workload, endpoints, realistic data, concurrency, ramp-up, assertions, latency/error thresholds, environment capacity and monitoring. The JD lists JMeter/Gatling/Locust as accepted tools.

Practical example:

JMeter plan:
Thread Group -> HTTP Request -> Header/Auth -> Assertions -> Listeners
Run non-GUI in CI and correlate results with CPU, memory and DB metrics.

Q104. What security testing should an SDET know?

Interview-ready answer: At minimum understand authentication/authorisation, input validation, secrets handling, session management, injection risks, sensitive-data exposure and OWASP-style controls. Do not replace specialist security testing.

Practical example:

Negative API:
No token -> 401
Wrong role -> 403
Unexpected input -> validation error
Sensitive payment data -> verify it is not exposed in logs.

Q105. How do you test production incidents?

Interview-ready answer: Reproduce safely, gather logs/metrics/traces, identify scope and customer impact, compare recent changes, create a minimal regression test, validate the fix and add monitoring/alerts where appropriate.

Practical example:

Incident: payment status stuck PROCESSING
→ trace transaction ID
→ inspect service/dependency logs
→ reproduce in lower environment
→ add regression/API test
→ verify deployment and monitoring.

Q106. What is regression vs retesting?

Interview-ready answer: Retesting verifies a specific defect fix. Regression checks that existing functionality has not been broken by changes.

Practical example:

Bug #123 fixed:
Retest -> exact failing scenario.
Regression -> payment creation, refund, reporting and related flows.

I. Payments / Financial Services / Kafka / Domain Questions

Q107. Explain a typical card payment flow.

Interview-ready answer: A simplified flow is customer → merchant/payment interface → gateway/acquirer → card scheme → issuer → authorisation response → capture/clearing/settlement. Exact architecture varies.

Practical example:

Test checkpoints:
request accepted → authentication/authorisation → response → transaction persistence → capture/settlement → merchant reporting

Q108. What is merchant acquiring?

Interview-ready answer: Acquiring is the service that enables merchants to accept payments; the acquirer connects to payment schemes, handles transaction processing and ultimately settlement. Worldline describes its acquiring services as covering authorisation, routing, capture/clearing and settlement activities.

Practical example:

Example:

Merchant submits card transaction → acquirer routes/authorises → transaction is captured → funds are cleared/settled to merchant

Q109. What is the difference between issuer and acquirer?

Interview-ready answer: The issuer is the financial institution associated with the cardholder/account and approves or declines based on its checks. The acquirer supports the merchant side and routes/processes transactions and settlement.

Practical example:

Cardholder bank = issuer

Merchant's acquiring institution/provider = acquirer

Q110. What is ISO 8583 and why might it matter?

Interview-ready answer: ISO 8583 is a message format commonly used for financial card transaction messaging. If the product uses it, an SDET may validate message types, fields, bitmaps/data elements and mappings.

Practical example:

Example test:

Authorization request → verify required transaction fields.

Response → map response code correctly to application status.

Q111. What are authorisation, capture, clearing and settlement?

Interview-ready answer: Authorisation checks/approves a transaction; capture confirms the amount for processing; clearing exchanges transaction data between participants; settlement moves funds.

Practical example:

State flow example:

AUTHORISED → CAPTURED → CLEARED → SETTLED

Also test DECLINED, CANCELLED and REFUNDED paths.

Q112. How would you test duplicate payment requests?

Interview-ready answer: Use the same idempotency key/reference, retry after simulated timeout, verify one business transaction, consistent response and correct ledger/settlement state.

Practical example:

POST payment with key K1

Network timeout

Retry POST with K1

Expected: no duplicate transaction.

Q113. How would you test a payment timeout?

Interview-ready answer: Simulate a slow/unavailable downstream dependency. Verify timeout handling, transaction state, client response, retry behaviour, idempotency and eventual reconciliation.

Practical example:

Gateway timeout:

Client -> 504/defined error

Backend transaction -> PENDING/UNKNOWN as contract defines

Retry -> safe, no duplicate charge.

Q114. What payment failure scenarios would you automate?

Interview-ready answer: Declined card, invalid merchant, invalid amount/currency, authentication failure, insufficient funds, duplicate request, timeout, downstream 5xx/502, partial failure, refund failure and reconciliation mismatch.

Practical example:

Data-driven test:

status=DECLINED

status=TIMEOUT

status=APPROVED

status=REFUNDED

Q115. Where can Kafka fit into a payment architecture?

Interview-ready answer: Kafka can be used for asynchronous event streaming such as transaction events, status updates or downstream processing. Your resume does not list Kafka hands-on experience, while the JD lists it as an advantage, so present this as conceptual unless you have additional experience.

Practical example:

```
Conceptual test:
payment.created event
→ consumer processes event
→ payment.status.updated
Verify ordering/idempotency/duplicate event handling where required.
```

Q116. How would you test an asynchronous payment flow?

Interview-ready answer: Validate API acknowledgement, event/message publication, eventual state transition, consumer processing, retries/dead-letter behaviour, idempotency and final persistence/reconciliation.

Practical example:

```
POST payment -> 202 Accepted
Poll/query status -> PROCESSING
Event consumed -> APPROVED
DB/report -> APPROVED
```

J. Scenario / Leadership / Behavioural Questions

Q117. A developer says your defect is not reproducible. What do you do?

Interview-ready answer: Provide exact environment/build, test data, timestamp, request/transaction ID, logs, screenshots/video, reproduction steps and compare configurations. Pair with the developer rather than debating ownership.

Practical example:

```
Bug evidence:
Build=1.8.12
Env=QA
Transaction=TX123
Steps=1..6
Expected=APPROVED
Actual=DECLINED
Attach API request/response + server log correlation ID.
```

Q118. Your UI regression has 20% flaky failures. What do you do?

Interview-ready answer: Measure failure patterns, classify root causes, remove hard waits, improve locators/synchronisation, isolate test data, fix environment dependencies and use targeted retries only where justified.

Practical example:

```
Dashboard:
40 failures → 25 timing → 10 data collision → 5 product defects.
Fix timing/data causes first; don't simply add retry=3.
```

Q119. Release is tomorrow and 30% of tests are failing. What is your approach?

Interview-ready answer: Do risk-based triage: distinguish product defects from infrastructure/test failures, prioritise critical business flows, run focused smoke/regression, communicate known risks and make a release recommendation based on evidence.

Practical example:

```
Priority:
Payment authorisation failure = immediate blocker/risk.
Cosmetic report alignment issue = lower release risk if not business critical.
```

Q120. Requirement is unclear but development has started. What do you do?

Interview-ready answer: Clarify acceptance criteria with product/development, document assumptions, identify examples and edge cases, and avoid automating ambiguous behaviour as if it were final.

Practical example:

```
Ask:
What is the expected status for duplicate payment?
What happens after timeout?
What are valid currencies?
What is the retry/idempotency contract?
```

Q121. How do you handle disagreement with a developer/product owner?

Interview-ready answer: Use evidence and agreed acceptance criteria. Reproduce, quantify impact, discuss alternatives, document the decision and escalate only when needed.

Practical example:

Instead of "this is wrong," say:
"Acceptance criterion X says Y. Current response is Z. Here is the reproducible request and business impact."

Q122. Tell me about a challenging automation migration.

Interview-ready answer: Use your actual Selenium-to-Playwright migration. Explain legacy pain points, target design, migration strategy, compatibility decisions, CI rollout, flaky tests and how you measured/observed improvement.

Practical example:

Structure:
Problem → constraints → design → migration in slices → parallel validation → CI integration → stabilisation → retirement of

Q123. How do you review automation code?

Interview-ready answer: Check correctness, readability, duplication, locator quality, synchronisation, assertions, test isolation, error handling, data cleanup, parallel safety and maintainability.

Practical example:

PR checklist:
No hard sleep
Stable locator
Business-level page method
Meaningful assertion
Independent data
Useful failure evidence

Q124. How do you mentor junior SDETs?

Interview-ready answer: Start with framework conventions, pair on real stories, explain why rather than only what, review PRs, assign progressively difficult work and create reusable examples/checklists.

Practical example:

Example:
Junior builds API test → review status/schema/data assertions → next task adds negative cases → later contributes a reusable A

Q125. What KPIs would you track for an SDET team?

Interview-ready answer: Use meaningful engineering/quality indicators rather than raw test counts: escaped defects, automation reliability/flakiness, feedback time, critical-path coverage, pipeline pass rate, defect detection stage, execution duration and MTTR where relevant.

Practical example:

Example dashboard:
Critical-flow automation coverage
Pipeline duration
Flaky-test rate
Escaped defects
Mean time to diagnose failed test

Q126. What would you ask the Worldline interviewer?

Interview-ready answer: Ask questions that reveal architecture, ownership and engineering expectations.

Practical example:

Good questions:
1. What layers of the testing pyramid are currently automated?
2. What is the current Java automation stack?
3. How are CI quality gates enforced?
4. How are payment simulators/test environments managed?
5. What are the main reliability challenges today?
6. How does the SDET collaborate with developers/product/operations?

K. External Research & Source Notes

Worldline JD — The uploaded JD emphasises end-to-end quality ownership, Scrum, testing pyramid, code reviews, adversarial testing, CI/CD quality gates, Jenkins/GitLab, Java, SQL, API testing, performance/security, Jira/Confluence, Linux, and optional GCP/Kubernetes/Docker/Kafka.

Reported Worldline interview questions — Public interview-reporting sources include questions on introduction, project, POM, framework, StringBuffer, HashMap vs Hashtable, Jenkins pipeline, abstract class vs interface, polymorphism, JavaScript Executor, rerunning failed tests, exceptions, Java final/finally and SQL primary key. These are treated as reported/community-sourced prompts, not guarantees of the current interview.

Broader SDET interview sources — Current SDET interview guides commonly cover framework design, test strategy, coding, API, SQL, Agile, automation strategy, testing pyramid, risk and scenario questions.

Worldline payments context — Worldline's current public material describes merchant acquiring as accepting/routing/authorising/settling payments and provides acquiring APIs for payment/transaction data. Payment-domain questions in this guide are therefore included as role-relevant preparation, not as claims about the exact internal architecture of this vacancy.

Public interview reports and current SDET preparation sources used for expansion include Worldline/Worldline Group interview reports and broader SDET interview guides. These sources report themes and questions from individual candidates; they do not establish that the same questions will be asked in your interview.

Worldline's public acquiring material describes merchant acquiring as a service covering authorisation, routing, capture/clearing and settlement, and its developer portal exposes acquiring payment/transaction APIs. This is useful context for the payment-domain questions in this guide.

Final preparation order

1. Java + OOP + Collections + Streams + coding → **2.** Framework architecture/POM/parallel execution → **3.** API + REST Assured/Postman + HTTP → **4.** Jenkins/CI quality gates → **5.** SQL + Linux → **6.** Testing pyramid/risk/adversarial scenarios → **7.** Payments/acquiring concepts → **8.** Playwright/TypeScript/JS because they are prominent on your resume → **9.** Docker/Kubernetes/AWS/Git → **10.** Performance/security + behavioural/leadership.